

Introducció a RTL-SDR

1.1. Objectius:

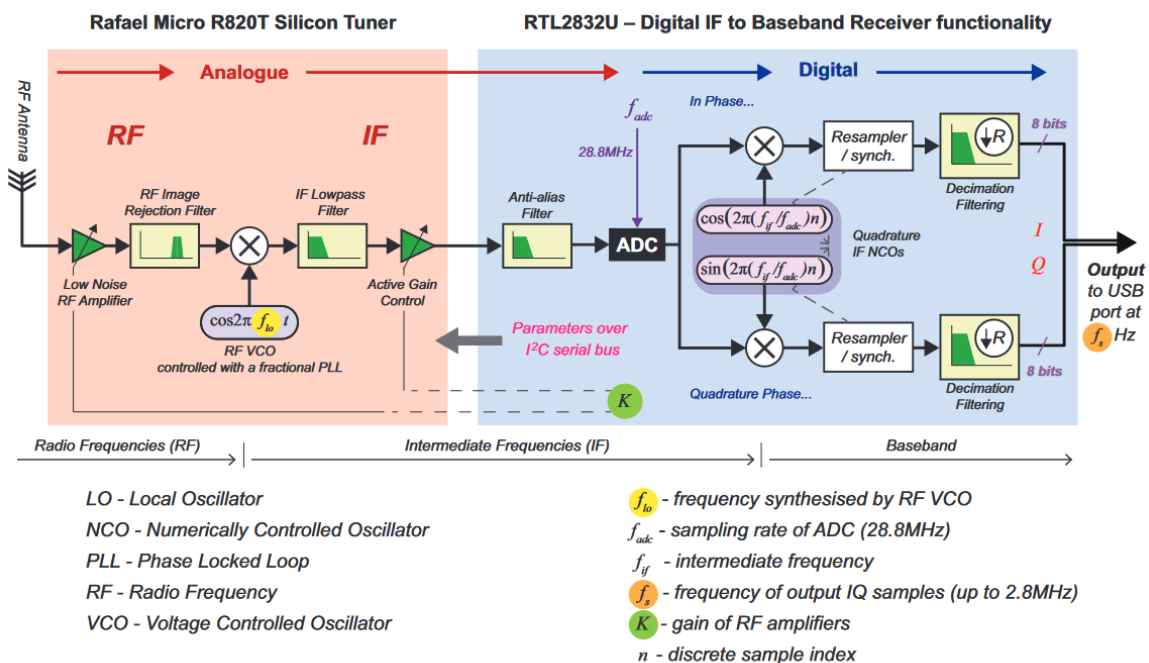
En aquesta sessió, introduïrem el dispositiu RTL-SDR i, amb Simulink, visualitzarem l'espectre corresponent a l'emissió d'FM. Per a fer això, primerament mirarem els diferents parts del RTL-SDR i les seves capacitats, després farem un petit model de Simulink on puguem veure l'espectre centrat al voltant de 90MHz (87.5MHz a 108MHz). Aquesta sessió de laboratori és semblant a l'anterior. Hi ha exercicis guiats, però, a cada tasca hi ha parts on haureu de fer feina.

1.2. Material:

- dispositiu RTL-SDR New Gen. RTL2832 de Nooelec
- Matlab & Simulink
- Communication Toolbox Support Package for RTL-SDR radio
- RTL-SDR Book Library

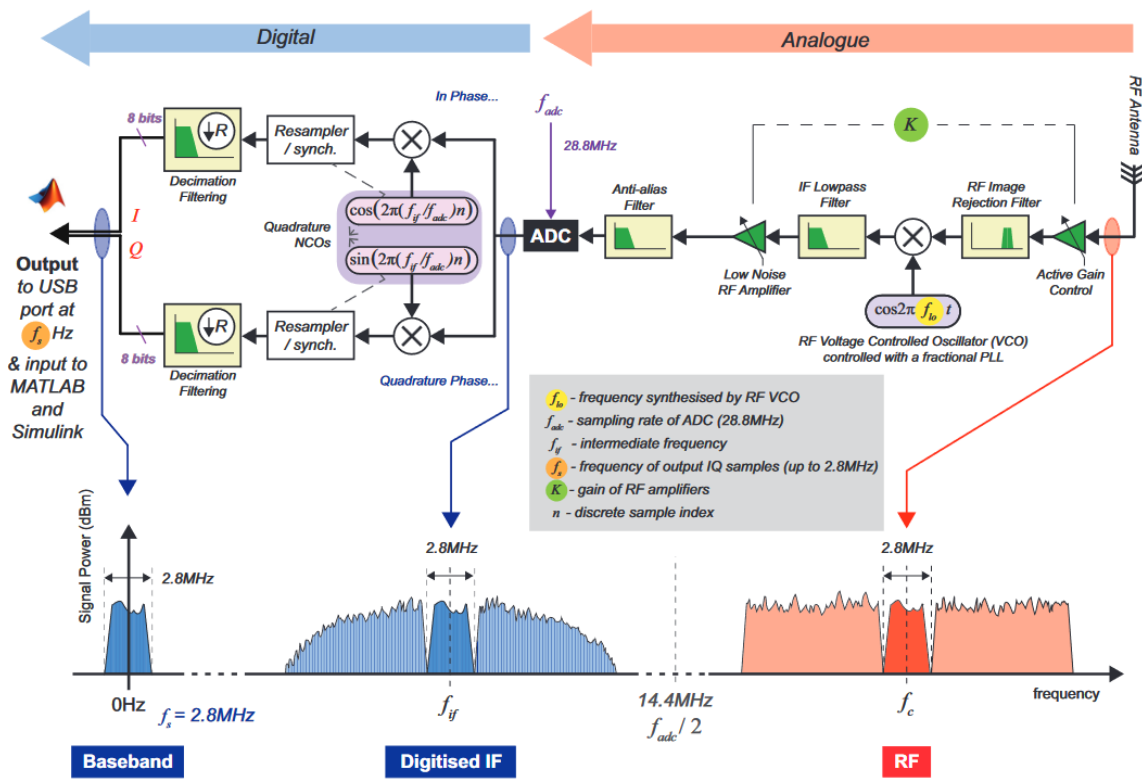
2. RTL-SDR

El dispositiu New Gen. RTL2832 de Nooelec té una estructura com la de la figura següent:



On podem veure la part de Radiofreqüència y la part de freqüència intermèdia com a part d'un equipament analògic, el filtre antialiàsing i la part de desmodulació I/Q després del convertidor A/D. La freqüència intermèdia és de 3.57MHz amb un amplada de banda de 6MHz, i la conversió A/D es fa a 28.8MHz. La desmodulació I/Q produeix dos senyals que es filtren amb una decimació de $L=10$, resultant una freqüència de mostreig final de 2.8MHz i paraules de 8-bit per nombre.

La figura següent mostra l'evolució de l'espectre quan el senyal va des de l'antena fins als senyals I/Q:



Tot això està contingut al petit dispositiu blau que teniu al costat de l'ordinador.

2. Tasques

a) Preparació de Matlab i Simulink.

En aquesta tasca confirmarem que Matlab reconeix el dispositiu RTL-SDR i que tenim les eines necessàries para generar models de Simulink amb aquest dispositiu.

a.1) Confirmació del SDR-RTL reconegut:

Feu connexió del RTL-SDR a qualsevol port USB de l'ordinador i aneu a Matlab.

Una vegada Matlab obert, a la línia de comandaments introduueix:

```
rtlSdr = sdrinfo
```

La resposta de Matlab ha de ser semblant al següent:

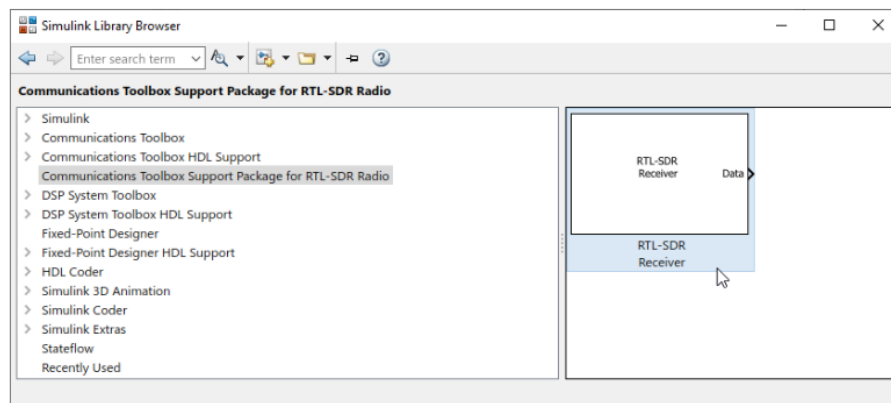
```
rtlsdr =
    RadioName: 'Generic RTL2832U OEM'
    RadioAddress: '0'
    RadioIsOpen: 0
    TunerName: 'R820T'
    Manufacturer: 'Realtek'
    Product: 'RTL2841UHIDIR'
    GainValues: [29x1 double]
    RTLCrystalFrequency: 28800000
    TunerCrystalFrequency: 28800000
    SamplingMode: 'Quadrature'
    OffsetTuning: 'Disabled'
```

a.2) RTL-SDR reconegut al Simulink:

Una vegada que hem vist que el RTL-SDR és reconegut per Matlab, mireu si el bloc del RTL-SDR està a la biblioteca de Simulink. Per això, a la línia de comandaments introduïm:

```
slLibraryBrowser
```

Aneu al **Communication Toolbox Support Package for RTL-SDR Radio** i fixeu-vos si surt un bloc com aquest:



A més, podeu veure si la documentació d'aquest paquet està instal·lada:

```
sdrrdoc
```

I la finestra d'ajuda ha de sortir.

Si tot ha anat bé, el dispositiu està preparat per les següents tasques. Si heu tingut qualsevol problema, contacteu amb el professor per fer una reinstal·lació del paquet de suport.

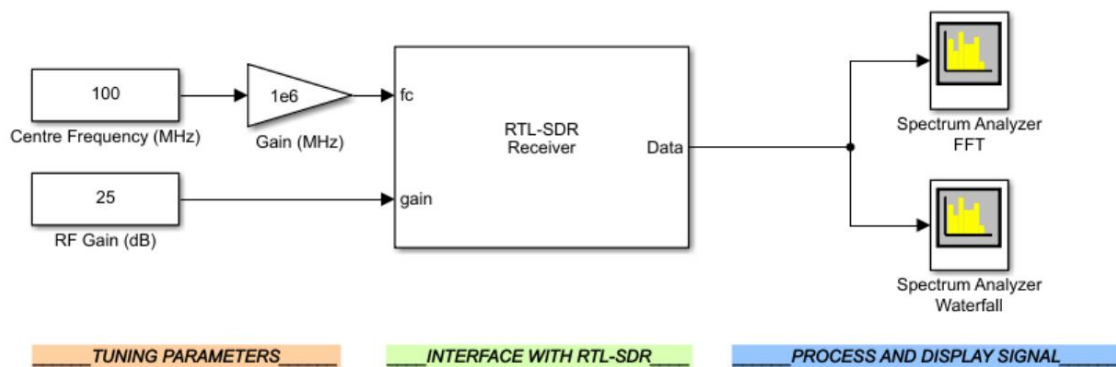
Si tot ha anat bé, el dispositiu està preparat per les següents tasques. Si heu tingut qualsevol problema, contacteu amb el professor per fer una re-instal·lació del paquet de suport.

b) Generació del Model Analitzador d'Espectres.

En aquesta tasca generarem un model per la visualització de l'espectre d'FM. També intentarem fer un escaneig no interactiu d'aquest espectre.

b.1) Generació del model:

Tenim que generar un model com aquest:



Heu de tenir en compte que el bloc RTL-SDR té una finestra de configuració com aquesta:

This is how the RTL-SDR Receiver block will appear in Simulink models

Double clicking on the block opens its parameters window.

tunable parameters

In the parameters window, you can configure the properties of your RTL-SDR

Block Parameters: RTL-SDR Receiver

RTL-SDR Receiver (mask) (link)

Receive data from an RTL-SDR radio.

Radio Connection

Radio address: 0

Info

Radio Configuration

Source of center frequency: Dialog

Center frequency (Hz): 100e6

Source of gain: Dialog

Tuner gain (dB): 25

Sampling rate (Hz): 2.4e6

Frequency correction (ppm): 0

Data Transfer Configuration

☐ Lost samples output port

☐ Latency output port

Output data type: single

Samples per frame: 4096

☐ Enable burst mode

OK Cancel Help Apply

En aquest cas, farem servir una freqüència de mostreig de 2.4MHz (una mica més baixa de 2.8MHz per seguretat) i heu de definir la freqüència central i el guany des de un input port. Si la freqüència de mostreig és de 2.4MHz, quins seran els límits de la nostra visualització en l'analitzador d'espectres? Quantes freqüències centrals hem de fer servir per obtenir una imatge de l'espectre d'FM?

b.2) post-processat:

Una vegada que tenim el dispositiu RTL-SDR inclòs a dins d'un model de Simulink, podem accedir a les mostres I/Q i importar-les al *workspace* de Matlab. Amb els mètodes vists a la sessió anterior, importeu a una variable anomenada *iqSignals* com una estructura. Per a fer això, limiteu el temps de simulació a 1 segon. Si el nombre de mostres per *frame* està limitat a 4096, això hauria de resultar en una matriu de 586x4096. Per què? D'on surt aquest 586?

Fixeu-vos que les dades en *iqSignals* són nombres complexos. La part real correspon al senyal en fase (I) i la part imaginària al senyal en quadratura (q). Llavors, la densitat espectral de potència es pot calcular de la següent manera:

1) Càlcul de l'espectre :

- a. `Fs = 2.4e6;`
- b. `N = 4096;`
- c. `M = 586;`
- d. `spectra = out.iqSignals.signals.values;`
- e. `fftSpectra = fftshift(fft(spectra,N,2));`
- f. `absSpectra = sum(abs(fftSpectra),1)./M;`

2) Càlcul de la densitat espectral de potència:

- a. `Psd = (1/(Fs*N))*absSpectra.^2;`
- b. `freqAxis = -Fs/2:FsWith(N-1):Fs/2;`

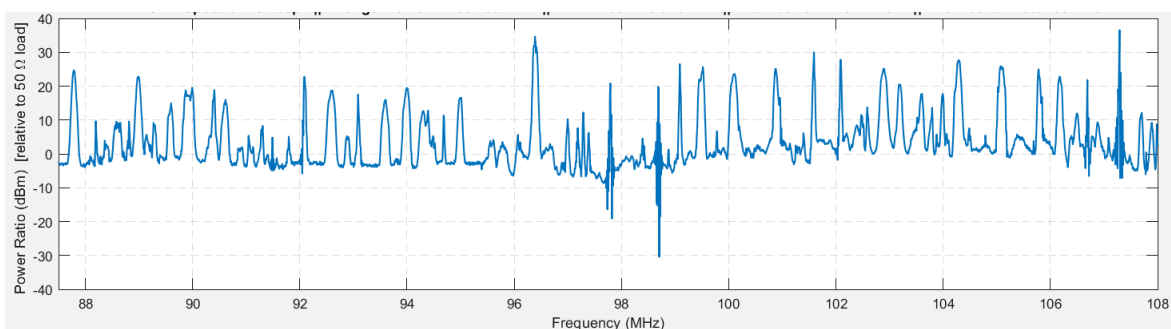
3) Visualització:

- a. `figure;`
- b. `plot(freqAxis, 10*log10(Psd/50));`

Fixeu-vos que hem fet el valor absolut mitjà de tot el senyal importat al *workspace* (1.e i 1.f). Després fem el càlcul de la densitat espectral de potència sobre aquest espectre mitjà. Quines diferències trobeu amb la representació de Simulink?

b.3) Visualització de l'espectre d'FM:

Si volem fer una visualització de l'espectre d'FM sencer, necessitareu importar al *workspace* de Matlab totes les porcions d'espectre que heu previst a l'apartat b.1. Per cadascuna dels senyals I/Q corresponents, heu de calcular la seva densitat espectral de potència i, quan tingueu totes les densitats heu d'afegir-les a una densitat sencera. De tal mena, que tindreu un espectre semblant a la següent figura:



c) Generació del script Analitzador d'Espectres.

En aquesta tasca mirarem de repetir el treball fet pel nostre model de Simulink, però fent servir una funció de MATLAB. Quan volem construir un sistema de recepció de radio amb el nostre receptor RTLSDR, el que tenim que fer és construir una funció en MATLAB que tingui el següent aspecte:

```
function P2_matlabCode

%% PARÀMETRES

% Aquí definim tots els paràmetres necessaris per a declarar els objectes i per a
% dur a terme la simulació
rtlsdr_id          = '0';           % RTL-SDR ID
rtlsdr_tunerfreq   = 100e6;         % RTL-SDR tuner frequency in Hz
rtlsdr_gain        = 25;            % RTL-SDR tuner gain in dB
rtlsdr_fs          = 2.4e6;         % RTL-SDR sampling rate in Hz
rtlsdr_frmlen      = 4096;          % RTL-SDR output data frame size
rtlsdr_datatype    = 'single';      % RTL-SDR output data type
rtlsdr_ppm         = 0;             % RTL-SDR tuner parts per million correction
sim_time          = 60;             % simulation time in seconds

%% OBJECTES DE SISTEMA

% Aquí declararem els objectes que té el nostre model. Al nostre cas, veiem que
% tenim tres objectes:
% 1) El receptor RTLSDR.
% 2) El analitzador d'espectres on es fa la FFT.
% 3) El analitzador d'espectres on es fa el waterfall.

% objecte RTLSDR.
obj_rtlsdr = comm.SDRRTLReceiver(...
    rtlsdr_id,...
    'CenterFrequency', rtlsdr_tunerfreq,...
    'EnableTunerAGC', false,...
    'TunerGain', rtlsdr_gain,...
    'SampleRate', rtlsdr_fs, ...
    'SamplesPerFrame', rtlsdr_frmlen,...
    'OutputDataType', rtlsdr_datatype ,...
    'FrequencyCorrection', rtlsdr_ppm );

% Analitzador d'espectres on es fa la FFT.
obj_specfft = dsp.SpectrumAnalyzer(...
    'Name', 'Spectrum Analyzer FFT',...
    'Title', 'Spectrum Analyzer FFT',...
    'SpectrumType', 'Power density',...
    'FrequencySpan', 'Full',...
    'SampleRate', rtlsdr_fs);

% Analitzador d'espectres on es fa el waterfall.
obj_specwaterfall = dsp.SpectrumAnalyzer(...
    'Name', 'Spectrum Analyzer Waterfall',...
    'Title', 'Spectrum Analyzer Waterfall',...
    'SpectrumType', 'Spectrogram',...
    'FrequencySpan', 'Full',...
    'SampleRate', rtlsdr_fs);
```

```
%% CÀLCULS
```

```
% Aquí es fan els càlculs necessaris per a dur a terme la simulació. Un dels
càlculs que es fa sempre és, el temps que triga un frame de dades en sortir del
receptor RTLSDR.
```

```
rtlsdr_frmtime = rtlsdr_frmlen/rtlsdr_fs;
```

```
%% SIMULACIÓ
```

```
% Sempre hem de comprovar que el receptor RTLSDR és actiu.
```

```
if isempty(sdrinfo(obj_rtlsdr.RadioAddress))
    error(['RTL-SDR failure. Please check connection to ',...
        'MATLAB using the "sdrinfo" command.']);
end
```

```
% Fem un reset del temps de simulació 0 (secs)
```

```
run_time = 0;
```

```
% Fins que el temps de simulació no arribi fins la durada de simulació definida
no parem de treure frames del receptor RTLSDR
```

```
while run_time < sim_time
```

```
    % Obtenim un frame
```

```
    rtlsdr_data = step(obj_rtlsdr);
```

```
    % Fem Update dels objectes de visualització
```

```
    step(obj_specfft, rtlsdr_data);
```

```
    step(obj_specwaterfall, rtlsdr_data);
```

```
    % Fem Update del temps de simulació
```

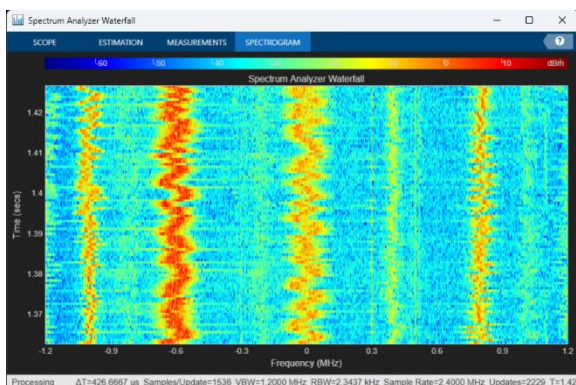
```
    run_time = run_time + rtlsdr_frmtime;
```

```
end
```

```
end %Acaba la funció. Ara si guardem aquesta funció al arxiu P2_matlabCode.
```

Ara, si guardem aquesta funció al arxiu P2_matlabCode.m, podem cridar quan vulguem al nostre analitzador de l'espectre radioelèctric. Fixeu-vos que les parts PARÀMETRES, OBJECTES, CÀLCULS i SIMULACIÓ, sempre hi seran a la funció del sistema. A més, el que canviarà d'un sistema a l'altre serà, per una banda els objectes i per l'altra la simulació (també els valors dels paràmetres i potser hem d'afegir algun paràmetres de més).

En tot cas, si fem servir aquest codi hauríem de veure dos analitzadors com aquests.



Quan el temps de simulació s'esgota, els analitzadors es tanquen. Anem a fer algunes cosetes amb aquesta funció.

c.1) Es demana modificar la funció `P2_matlabCode` per a generar una matriu de dades amb tots els *frames* corresponents a 1 segon de simulació. Podries generar la mateixa figura obtinguda l'apartat b.2)?

c.2) Una vegada podem trobar el PSD de un tros de l'espectre de FM. Podries modificar la funció `P2_matlabCode` per a escanejar tot l'espectre de FM i fer el càlcul del PSD? Compte que és més viable fer el càlcul del PSD per un tros i guardar només aquesta informació que no pas guardar totes les matrius de dades corresponents a cadascun dels trossos.